RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 1: Data Preprocessing and Cleaning

Name: Nischal **Date:** 06-10-2026

Register Number: 241801188 **Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn

Dataset: Telco Customer Churn (IBM)

Aim

To preprocess the Telco Customer Churn dataset by handling missing values, removing inconsistencies and duplicates, encoding categorical features, standardising numerical attributes and visualising important relationships, so that the data is ready for building a churn prediction model.

Task 1: Import the required Python libraries

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, StandardScaler

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid')
pd.set_option('display.max_columns', None)

print("Libraries imported successfully")
print("pandas:", pd.__version__, "| numpy:", np.__version__, "| seaborn:", sns.__version__)
```

Out [1]:

```
Libraries imported successfully
pandas: 3.0.5 | numpy: 2.5.3 | seaborn: 0.13.2
```

Task 2: Load the Telco Customer Churn dataset into a Pandas DataFrame

In [2]:

```
df = pd.read_csv('Telco-Customer-Churn.csv')
print("Dataset loaded successfully")
```

Out [2]:

```
Dataset loaded successfully
```

2.1 First 10 records

In [3]:

```
df.head(10)
```

Out [3]:

Columns 1–8 of 21

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service
1	5575-GNVDE	Male	0	No	No	34	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No
3	7795-CFOCW	Male	0	No	No	45	No	No phone service
4	9237-HQITU	Female	0	No	No	2	Yes	No
5	9305-CDSKC	Female	0	No	No	8	Yes	Yes
6	1452-KIOVK	Male	0	No	Yes	22	Yes	Yes
7	6713-OKOMC	Female	0	No	No	10	No	No phone service
8	7892-POOKP	Female	0	Yes	No	28	Yes	Yes
9	6388-TABGU	Male	0	No	Yes	62	Yes	No

Columns 9–14 of 21

	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV
0	DSL	No	Yes	No	No	No
1	DSL	Yes	No	Yes	No	No
2	DSL	Yes	Yes	No	No	No
3	DSL	Yes	No	Yes	Yes	No
4	Fiber optic	No	No	No	No	No
5	Fiber optic	No	No	Yes	No	Yes
6	Fiber optic	No	Yes	No	No	Yes
7	DSL	Yes	No	No	No	No
8	Fiber optic	No	No	Yes	Yes	Yes
9	DSL	Yes	Yes	No	No	No

Columns 15–19 of 21

	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges
0	No	Month-to-month	Yes	Electronic check	29.85
1	No	One year	No	Mailed check	56.95
2	No	Month-to-month	Yes	Mailed check	53.85
3	No	One year	No	Bank transfer (automatic)	42.30
4	No	Month-to-month	Yes	Electronic check	70.70
5	Yes	Month-to-month	Yes	Electronic check	99.65
6	No	Month-to-month	Yes	Credit card (automatic)	89.10
7	No	Month-to-month	No	Mailed check	29.75
8	Yes	Month-to-month	Yes	Electronic check	104.80
9	No	One year	No	Bank transfer (automatic)	56.15

Columns 20–21 of 21

	TotalCharges	Churn
0	29.85	No
1	1889.5	No
2	108.15	Yes
3	1840.75	No
4	151.65	Yes
5	820.5	Yes
6	1949.4	No
7	301.9	No
8	3046.05	Yes
9	3487.95	No

10 rows × 21 columns

2.2 Last 10 records

In [4]:

```
df.tail(10)
```

Out [4]:

Columns 1–7 of 21

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService
7033	9767-FFLEM	Male	0	No	No	38	Yes
7034	0639-TSIQW	Female	0	No	No	67	Yes
7035	8456-QDAVC	Male	0	No	No	19	Yes
7036	7750-EYXWZ	Female	0	No	No	12	No
7037	2569-WGERO	Female	0	No	No	72	Yes
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes
7040	4801-JZAZL	Female	0	Yes	Yes	11	No
7041	8361-LTMKD	Male	1	Yes	No	4	Yes
7042	3186-AJIEK	Male	0	No	No	66	Yes

Columns 8–11 of 21

	MultipleLines	InternetService	OnlineSecurity	OnlineBackup
7033	No	Fiber optic	No	No
7034	Yes	Fiber optic	Yes	Yes
7035	No	Fiber optic	No	No
7036	No phone service	DSL	No	Yes
7037	No	No	No internet service	No internet service
7038	Yes	DSL	Yes	No
7039	Yes	Fiber optic	No	Yes
7040	No phone service	DSL	Yes	No
7041	Yes	Fiber optic	No	No
7042	No	Fiber optic	Yes	No

Columns 12–15 of 21

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies
7033	No	No	No	No
7034	Yes	No	Yes	No
7035	No	No	Yes	No
7036	Yes	Yes	Yes	Yes
7037	No internet service	No internet service	No internet service	No internet service
7038	Yes	Yes	Yes	Yes
7039	Yes	No	Yes	Yes
7040	No	No	No	No
7041	No	No	No	No
7042	Yes	Yes	Yes	Yes

Columns 16–20 of 21

	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges
7033	Month-to-month	Yes	Credit card (automatic)	69.50	2625.25
7034	Month-to-month	Yes	Credit card (automatic)	102.95	6886.25
7035	Month-to-month	Yes	Bank transfer (automatic)	78.70	1495.1
7036	One year	No	Electronic check	60.65	743.3
7037	Two year	Yes	Bank transfer (automatic)	21.15	1419.4
7038	One year	Yes	Mailed check	84.80	1990.5
7039	One year	Yes	Credit card (automatic)	103.20	7362.9
7040	Month-to-month	Yes	Electronic check	29.60	346.45
7041	Month-to-month	Yes	Mailed check	74.40	306.6
7042	Two year	Yes	Bank transfer (automatic)	105.65	6844.5

Columns 21–21 of 21

	Churn
7033	No
7034	Yes
7035	No
7036	No
7037	No
7038	No
7039	No
7040	No
7041	Yes
7042	No

10 rows × 21 columns

2.3 Shape of the dataset

In [5]:

```
print("Shape of the dataset:", df.shape)
print("Number of rows :", df.shape[0])
print("Number of columns:", df.shape[1])
```

Out [5]:

```
Shape of the dataset: (7043, 21)
Number of rows : 7043
Number of columns: 21
```

2.4 Column names

In [6]:

```
print("Column names:")
for i, col in enumerate(df.columns, start=1):
    print(f"{i:2d}. {col}")
```

Out [6]:

```
Column names:
 1. customerID
 2. gender
 3. SeniorCitizen
 4. Partner
 5. Dependents
 6. tenure
 7. PhoneService
 8. MultipleLines
 9. InternetService
10. OnlineSecurity
11. OnlineBackup
12. DeviceProtection
13. TechSupport
14. StreamingTV
15. StreamingMovies
16. Contract
17. PaperlessBilling
18. PaymentMethod
19. MonthlyCharges
20. TotalCharges
21. Churn
```

Task 3: Explore the dataset

3.1 Dataset information

In [7]:

```
df.info()
```

Out [7]:

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   str    
 1   gender            7043 non-null   str    
 2   SeniorCitizen     7043 non-null   int64  
 3   Partner           7043 non-null   str    
 4   Dependents        7043 non-null   str    
 5   tenure            7043 non-null   int64  
 6   PhoneService      7043 non-null   str    
 7   MultipleLines     7043 non-null   str    
 8   InternetService   7043 non-null   str    
 9   OnlineSecurity    7043 non-null   str    
 10  OnlineBackup      7043 non-null   str    
 11  DeviceProtection  7043 non-null   str    
 12  TechSupport       7043 non-null   str    
 13  StreamingTV       7043 non-null   str    
 14  StreamingMovies   7043 non-null   str    
 15  Contract          7043 non-null   str    
 16  PaperlessBilling  7043 non-null   str    
 17  PaymentMethod     7043 non-null   str    
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   str    
 20  Churn             7043 non-null   str    
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

3.2 Summary statistics

In [8]:

```
# Summary statistics of numerical columns
df.describe()
```

Out [8]:

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

8 rows × 3 columns

In [9]:

```
# Summary statistics of categorical columns
df.describe(exclude='number').T
```

Out [9]:

	count	unique	top	freq
customerID	7043	7043	7590-VHVEG	1
gender	7043	2	Male	3555
Partner	7043	2	No	3641
Dependents	7043	2	No	4933
PhoneService	7043	2	Yes	6361
MultipleLines	7043	3	No	3390
InternetService	7043	3	Fiber optic	3096
OnlineSecurity	7043	3	No	3498
OnlineBackup	7043	3	No	3088
DeviceProtection	7043	3	No	3095
TechSupport	7043	3	No	3473
StreamingTV	7043	3	No	2810
StreamingMovies	7043	3	No	2785
Contract	7043	3	Month-to-month	3875
PaperlessBilling	7043	2	Yes	4171
PaymentMethod	7043	4	Electronic check	2365
TotalCharges	7043	6531	20.2	11
Churn	7043	2	No	5174

18 rows × 4 columns

3.3 Number of missing values

In [10]:

```
missing = df.isnull().sum()
print("Missing values in each column (as read from the file):")
print(missing)
print("\nTotal missing values:", missing.sum())
```

Out [10]:

```
Missing values in each column (as read from the file):
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Total missing values: 0
```

Checking for hidden inconsistencies. `TotalCharges` holds amounts but was read as text, which means some rows contain a non-numeric value. These blanks are not counted by `isnull()`.

In [11]:

```
blank_rows = df[df['TotalCharges'].str.strip() == '']
print("Rows with a blank TotalCharges value:", len(blank_rows))
display(blank_rows[['customerID', 'tenure', 'MonthlyCharges', 'TotalCharges']])
```

Out [11]:

Rows with a blank TotalCharges value: 11

	customerID	tenure	MonthlyCharges	TotalCharges
488	4472-LVYGI	0	52.55	
753	3115-CZMZD	0	20.25	
936	5709-LVOEQ	0	80.85	
1082	4367-NUYAO	0	25.75	
1340	1371-DWPAZ	0	56.05	
3331	7644-OMVMY	0	19.85	
3826	3213-VVOLG	0	25.35	
4380	2520-SGTTA	0	20.00	
5218	2923-ARZLG	0	19.70	
6670	4075-WKNIU	0	73.35	
6754	2775-SEFEE	0	61.90	

11 rows × 4 columns

In [12]:

```
# Fix: remove extra spaces from text columns and convert TotalCharges to numeric.
# Blanks become NaN so that they are treated as missing values.
text_cols = df.select_dtypes(exclude='number').columns
df[text_cols] = df[text_cols].apply(lambda s: s.str.strip())
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
missing = df.isnull().sum()
print("Missing values after correcting the data type:")
print(missing[missing > 0])
print("\nTotal missing values:", missing.sum())
```

Out [12]:

```
Missing values after correcting the data type:
TotalCharges    11
dtype: int64

Total missing values: 11
```

> **Observation:** All 11 rows with a blank `TotalCharges` have `tenure = 0`; they are new customers who have not been billed yet. After conversion these blanks become missing values, which are handled in Task 4.

In [13]:

```
# Check the categories of the columns to be encoded for inconsistent labels
for col in ['gender', 'Partner', 'Dependents', 'Contract', 'PaymentMethod', 'InternetService', 'Churn']:
    print(f"{col:16s}: {df[col].unique().tolist()}")
```

Out [13]:

```
gender          : ['Female', 'Male']
Partner         : ['Yes', 'No']
Dependents      : ['No', 'Yes']
Contract        : ['Month-to-month', 'One year', 'Two year']
PaymentMethod   : ['Electronic check', 'Mailed check', 'Bank transfer (automatic)', 'Credit card (automatic)']
InternetService : ['DSL', 'Fiber optic', 'No']
Churn           : ['No', 'Yes']
```

3.4 Number of duplicate records

In [14]:

```
print("Duplicate rows (all columns):", df.duplicated().sum())
print("Duplicate customerID values :", df['customerID'].duplicated().sum())
```

Out [14]:

```
Duplicate rows (all columns): 0
Duplicate customerID values : 0
```

Task 4: Clean the data

4(a) Remove duplicate customer records

In [15]:

```
rows_before = df.shape[0]
df = df.drop_duplicates()                                    # exact duplicate rows
df = df.drop_duplicates(subset='customerID', keep='first')  # same customer listed twice
print("Rows before removing duplicates:", rows_before)
print("Rows after removing duplicates :", df.shape[0])
print("Duplicate records removed      :", rows_before - df.shape[0])
```

Out [15]:

```
Rows before removing duplicates: 7043
Rows after removing duplicates : 7043
Duplicate records removed      : 0
```

4(b) Replace missing values (numerical → median, categorical → mode)

In [16]:

```
num_cols = df.select_dtypes(include='number').columns
cat_cols = df.select_dtypes(exclude='number').columns

# i. Numerical columns -> Median
for col in num_cols:
    n_missing = df[col].isnull().sum()
    if n_missing > 0:
        median_value = df[col].median()
        df[col] = df[col].fillna(median_value)
        print(f"[Numerical] {col}: {n_missing} missing values replaced with median = {median_value:.2f}")

# ii. Categorical columns -> Mode
for col in cat_cols:
    n_missing = df[col].isnull().sum()
    if n_missing > 0:
        mode_value = df[col].mode()[0]
        df[col] = df[col].fillna(mode_value)
        print(f"[Categorical] {col}: {n_missing} missing values replaced with mode = {mode_value}")

print("\nNumerical columns :", list(num_cols))
print("Categorical columns:", len(cat_cols), "columns checked")
```

Out [16]:

```
[Numerical] TotalCharges: 11 missing values replaced with median = 1397.47

Numerical columns : ['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges']
Categorical columns: 17 columns checked
```

4(c) Verify that no missing values remain

In [17]:

```
remaining = df.isnull().sum()
print(remaining)
print("\nTotal missing values remaining:", remaining.sum())
if remaining.sum() == 0:
    print("Verified: the dataset has no missing values.")

# Keep a copy of the cleaned data (original units) for the visualisations
df_clean = df.copy()
print("Cleaned dataset shape:", df_clean.shape)
```

Out [17]:

```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Total missing values remaining: 0
Verified: the dataset has no missing values.
Cleaned dataset shape: (7043, 21)
```

Task 5: Encode categorical columns

- **Label Encoding** for binary columns (two categories): `gender`, `Partner`, `Dependents`, `Churn`
- **Ordinal Encoding** for `Contract`, because its categories have a natural order (month-to-month < one year < two year)
- **One-Hot Encoding** for `PaymentMethod` and `InternetService`, because their categories have no order

In [18]:

```
df_encoded = df_clean.copy()

# (a) Label Encoding for binary columns
binary_cols = ['gender', 'Partner', 'Dependents', 'Churn']
le = LabelEncoder()
for col in binary_cols:
    df_encoded[col] = le.fit_transform(df_encoded[col])
    mapping = {cls: i for i, cls in enumerate(le.classes_)}
    print(f"{col:11s}: {mapping}")
```

Out [18]:

```
gender     : {'Female': 0, 'Male': 1}
Partner    : {'No': 0, 'Yes': 1}
Dependents : {'No': 0, 'Yes': 1}
Churn      : {'No': 0, 'Yes': 1}
```

In [19]:

```
# (b) Ordinal Encoding for Contract
contract_order = [['Month-to-month', 'One year', 'Two year']]
oe = OrdinalEncoder(categories=contract_order)
df_encoded['Contract'] = oe.fit_transform(df_encoded[['Contract']]).astype(int)
print("Contract :", {cat: i for i, cat in enumerate(contract_order[0])})
```

Out [19]:

```
Contract : {'Month-to-month': 0, 'One year': 1, 'Two year': 2}
```

In [20]:

```
# (c) One-Hot Encoding for nominal columns with more than two categories
df_encoded = pd.get_dummies(df_encoded, columns=['PaymentMethod', 'InternetService'], dtype=int)

onehot_cols = [c for c in df_encoded.columns if c.startswith(('PaymentMethod_', 'InternetService_'))]
print("New one-hot columns:")
for c in onehot_cols:
    print("  ", c)
print("\nShape after encoding:", df_encoded.shape)
```

Out [20]:

```
New one-hot columns:
   PaymentMethod_Bank transfer (automatic)
   PaymentMethod_Credit card (automatic)
   PaymentMethod_Electronic check
   PaymentMethod_Mailed check
   InternetService_DSL
   InternetService_Fiber optic
   InternetService_No

Shape after encoding: (7043, 26)
```

In [21]:

```
# Encoded columns - first 10 records
encoded_cols = binary_cols + ['Contract'] + onehot_cols
df_encoded[encoded_cols].head(10)
```

Out [21]:

Columns 1–6 of 12

	gender	Partner	Dependents	Churn	Contract	PaymentMethod_Bank transfer (automatic)
0	0	1	0	0	0	0
1	1	0	0	0	1	0
2	1	0	0	1	0	0
3	1	0	0	0	1	1
4	0	0	0	1	0	0
5	0	0	0	1	0	0
6	1	0	1	0	0	0
7	0	0	0	0	0	0
8	0	1	0	1	0	0
9	1	0	1	0	1	1

Columns 7–9 of 12

	PaymentMethod_Credit card (automatic)	PaymentMethod_Electronic check	PaymentMethod_Mailed check
0	0	1	0
1	0	0	1
2	0	0	1
3	0	0	0
4	0	1	0
5	0	1	0
6	1	0	0
7	0	0	1
8	0	1	0
9	0	0	0

Columns 10–12 of 12

	InternetService_DSL	InternetService_Fiber optic	InternetService_No
0	1	0	0
1	1	0	0
2	1	0	0
3	1	0	0
4	0	1	0
5	0	1	0
6	0	1	0
7	1	0	0
8	0	1	0
9	1	0	0

10 rows × 12 columns

Task 6: Standardise the numerical features using StandardScaler

In [22]:

```
scale_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']

print("Before scaling:")
display(df_encoded[scale_cols].describe().round(2))

scaler = StandardScaler()
df_encoded[scale_cols] = scaler.fit_transform(df_encoded[scale_cols])

print("After scaling (mean = 0, standard deviation = 1):")
display(df_encoded[scale_cols].describe().round(2))
```

Out [22]:

Before scaling:

	tenure	MonthlyCharges	TotalCharges
count	7043.00	7043.00	7043.00
mean	32.37	64.76	2281.92
std	24.56	30.09	2265.27
min	0.00	18.25	18.80
25%	9.00	35.50	402.22
50%	29.00	70.35	1397.48
75%	55.00	89.85	3786.60
max	72.00	118.75	8684.80

8 rows × 3 columns

After scaling (mean = 0, standard deviation = 1):

	tenure	MonthlyCharges	TotalCharges
count	7043.00	7043.00	7043.00
mean	-0.00	-0.00	-0.00
std	1.00	1.00	1.00
min	-1.32	-1.55	-1.00
25%	-0.95	-0.97	-0.83
50%	-0.14	0.19	-0.39
75%	0.92	0.83	0.66
max	1.61	1.79	2.83

8 rows × 3 columns

In [23]:

```
# Standardised features - first 10 records
df_encoded[scale_cols].head(10)
```

Out [23]:

	tenure	MonthlyCharges	TotalCharges
0	-1.277445	-1.160323	-0.994242
1	0.066327	-0.259629	-0.173244
2	-1.236724	-0.362660	-0.959674
3	0.514251	-0.746535	-0.194766
4	-1.236724	0.197365	-0.940470
5	-0.992402	1.159546	-0.645186
6	-0.422317	0.808907	-0.146799
7	-0.910961	-1.163647	-0.874137
8	-0.177995	1.330711	0.337349
9	1.206498	-0.286218	0.532439

10 rows × 3 columns

Task 7: Distribution of MonthlyCharges and Tenure

The plots use the cleaned data in original units (before scaling) so the axes are easy to read.

In [24]:

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))

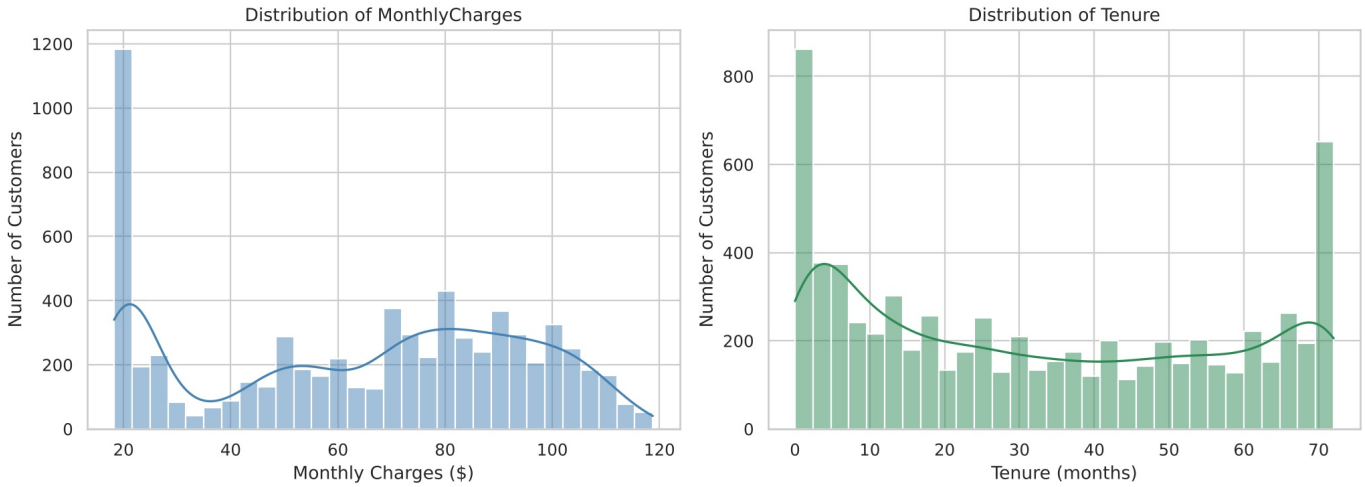
sns.histplot(df_clean['MonthlyCharges'], bins=30, kde=True, color='steelblue', ax=axes[0])
axes[0].set_title('Distribution of MonthlyCharges')
axes[0].set_xlabel('Monthly Charges ($)')
axes[0].set_ylabel('Number of Customers')

sns.histplot(df_clean['tenure'], bins=30, kde=True, color='seagreen', ax=axes[1])
axes[1].set_title('Distribution of Tenure')
axes[1].set_xlabel('Tenure (months)')
axes[1].set_ylabel('Number of Customers')

plt.tight_layout()
plt.show()

print("MonthlyCharges -> mean: %.2f, median: %.2f, skewness: %.2f"
      % (df_clean['MonthlyCharges'].mean(), df_clean['MonthlyCharges'].median(),
         df_clean['MonthlyCharges'].skew()))
print("Tenure         -> mean: %.2f, median: %.2f, skewness: %.2f"
      % (df_clean['tenure'].mean(), df_clean['tenure'].median(), df_clean['tenure'].skew()))
```

Out [24]:




```
MonthlyCharges -> mean: 64.76, median: 70.35, skewness: -0.22
Tenure         -> mean: 32.37, median: 29.00, skewness: 0.24
```

Observation: MonthlyCharges is spread widely: a large group of customers pay around $20 (basic plans), while most others pay $70–$100, so the mean (64.76) is below the median (70.35). Tenure is U-shaped: many customers are new (0–5 months) and many have stayed about 70 months.

Task 8: Count plot of Contract vs Churn

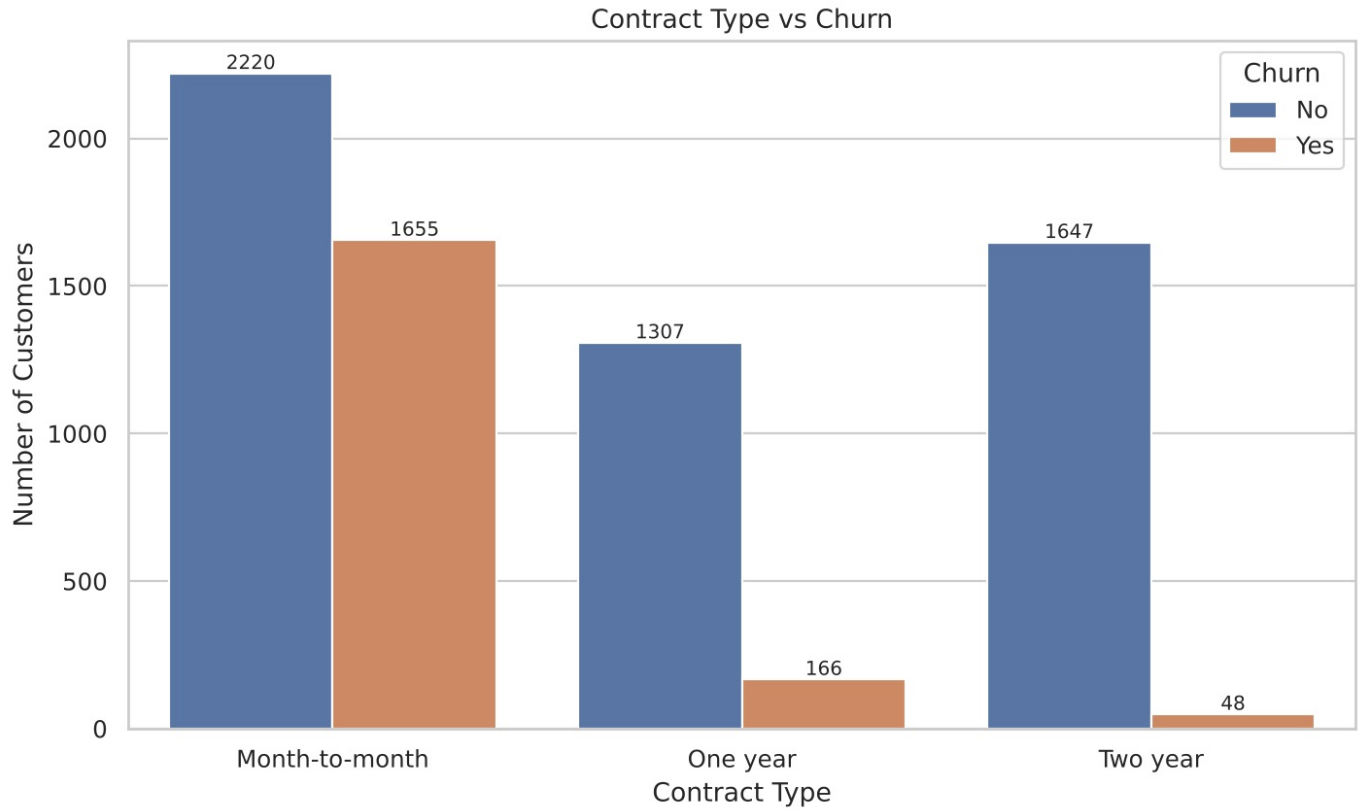
In [25]:

```
churn_palette = {'No': '#4C72B0', 'Yes': '#DD8452'}

plt.figure(figsize=(9, 5.5))
ax = sns.countplot(data=df_clean, x='Contract', hue='Churn', palette=churn_palette,
                   order=['Month-to-month', 'One year', 'Two year'], hue_order=['No', 'Yes'])
for container in ax.containers:
    ax.bar_label(container, fontsize=9)
plt.title('Contract Type vs Churn')
plt.xlabel('Contract Type')
plt.ylabel('Number of Customers')
plt.legend(title='Churn')
plt.tight_layout()
plt.show()

# Churn rate (%) within each contract type
churn_rate = pd.crosstab(df_clean['Contract'], df_clean['Churn'], normalize='index') * 100
print("Churn rate (%) by contract type:")
print(churn_rate.round(2))
```

Out [25]:




```
Churn rate (%) by contract type:
Churn              No    Yes
Contract
Month-to-month  57.29  42.71
One year        88.73  11.27
Two year        97.17   2.83
```

> **Observation:** Month-to-month customers churn far more (42.71%) than one-year (11.27%) and two-year (2.83%) customers, so longer contracts help retain customers.

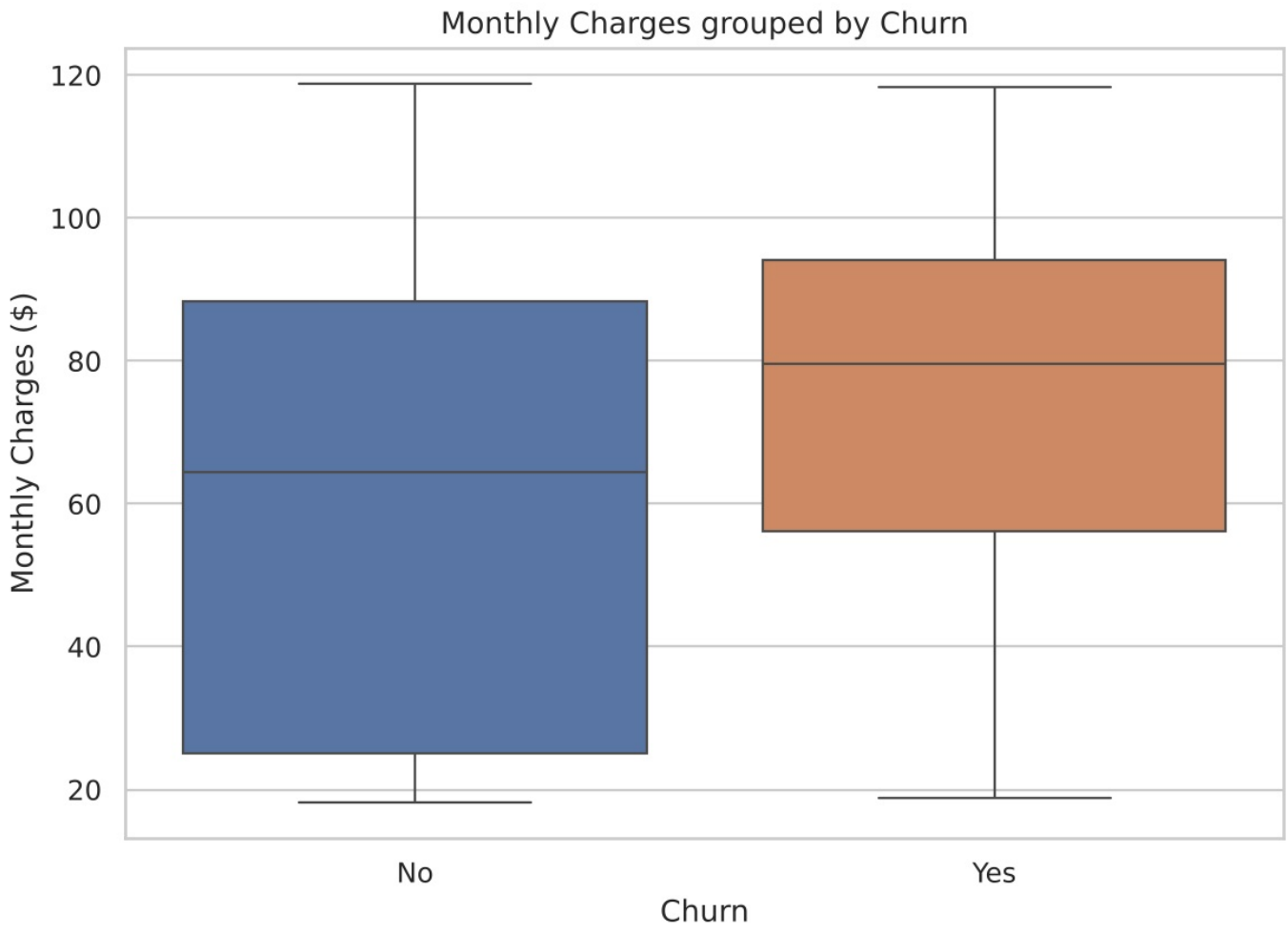
Task 9: Box plot of MonthlyCharges grouped by Churn

In [26]:

```
plt.figure(figsize=(7.5, 5.5))
sns.boxplot(data=df_clean, x='Churn', y='MonthlyCharges', hue='Churn',
            palette=churn_palette, order=['No', 'Yes'], legend=False)
plt.title('Monthly Charges grouped by Churn')
plt.xlabel('Churn')
plt.ylabel('Monthly Charges ($)')
plt.tight_layout()
plt.show()

print(df_clean.groupby('Churn')['MonthlyCharges'].describe().round(2))
```

Out [26]:



Churn	count	mean	std	min	25%	50%	75%	max
No	5174.0	61.27	31.09	18.25	25.10	64.43	88.4	118.75
Yes	1869.0	74.44	24.67	18.85	56.15	79.65	94.2	118.35

Observation: Customers who churned pay higher monthly charges (median $79.65) than customers who stayed (median $64.43).

Task 10: Correlation heatmap of all numerical features

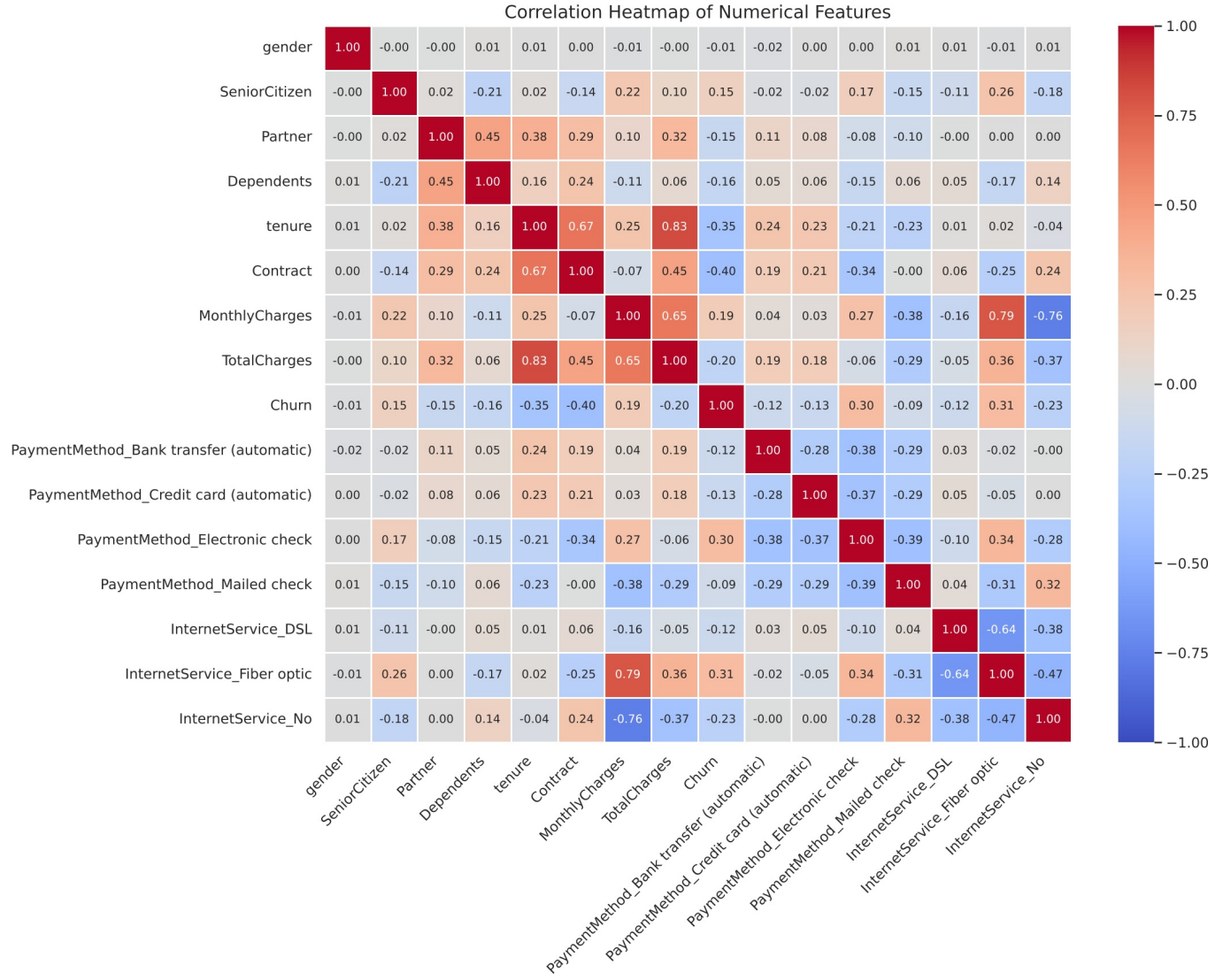
After encoding and scaling, every feature used in this experiment is numerical, so the heatmap covers all numerical columns of the preprocessed dataset.

In [27]:

```
numeric_df = df_encoded.select_dtypes(include='number')
corr = numeric_df.corr()

plt.figure(figsize=(14, 11))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, center=0,
            linewidths=0.5, annot_kws={'size': 9})
plt.title('Correlation Heatmap of Numerical Features', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Out [27]:



In [28]:

```
# Correlation of each feature with Churn
print(corr['Churn'].drop('Churn').sort_values(ascending=False).round(3))
```

Out [28]:

```
InternetService_Fiber optic               0.308
PaymentMethod_Electronic check            0.302
MonthlyCharges                            0.193
SeniorCitizen                             0.151
gender                                   -0.009
PaymentMethod_Mailed check               -0.092
PaymentMethod_Bank transfer (automatic)  -0.118
InternetService_DSL                      -0.124
PaymentMethod_Credit card (automatic)    -0.134
Partner                                  -0.150
Dependents                               -0.164
TotalCharges                             -0.199
InternetService_No                       -0.228
tenure                                   -0.352
Contract                                 -0.397
Name: Churn, dtype: float64
```

> **Observation:** Churn is most negatively correlated with Contract (-0.40) and tenure (-0.35), and most positively with Fiber optic internet (0.31) and Electronic check payment (0.30). TotalCharges is strongly correlated with tenure (0.83), since it accumulates over time.

Result

Thus the Telco Customer Churn dataset (7,043 records, 21 attributes) was successfully preprocessed using Python. The data was explored, the hidden blank values in TotalCharges were converted to missing values and replaced with the median, duplicates were checked and removed, the categorical columns were encoded using label, ordinal and one-hot encoding, the numerical features were standardised using StandardScaler, and the key relationships were visualised. The cleaned dataset is ready for building a churn prediction model.